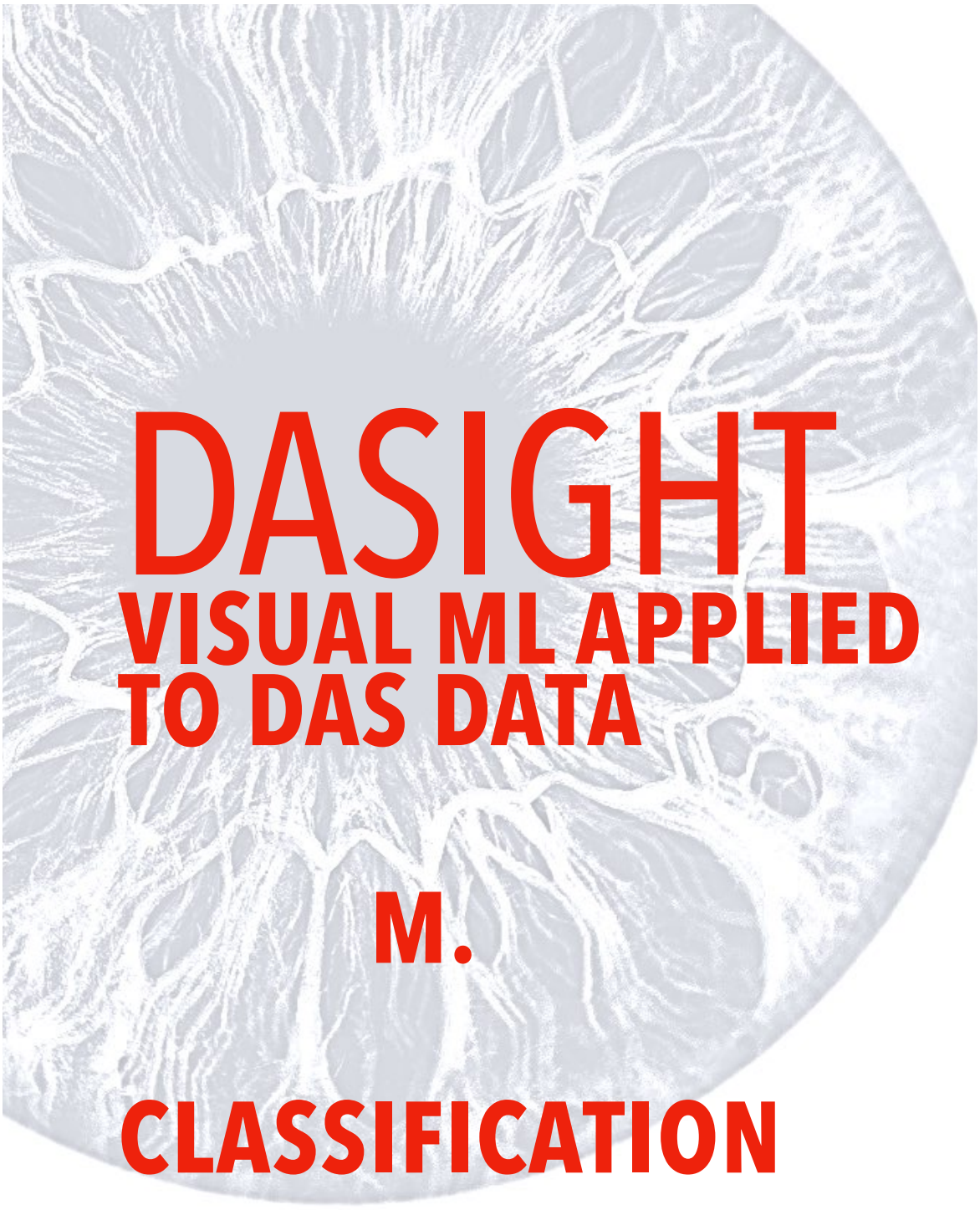


DASIGHT

VISUAL ML APPLIED TO DAS DATA

M.

CLASSIFICATION

DASIGHT MANUAL

VISUAL ML APPLIED TO DAS DATA CLASSIFICATION



MARIANO S. ARNAIZ-RODRÍGUEZ[1]

CLAUDIO SATRIANO[1]

[1]Université Paris Cité, Institut de physique du globe de Paris, CNRS, Paris F-75005, France

© Arnaiz-R., M. and Satriano, C. (2026)

PROLOGUE

To grant me a vision of Nature's forces
That bind the world, all its seeds and sources
And innermost life—all this I shall see,
And stop peddling in words that mean nothing to me.

Faust - J.W. von Goethe

Hi there!

If you are here, you are most likely dealing with some Distributed Acoustic Sensing (DAS) data. Moreover, you might be asking yourself if Machine Learning/Convolutional Neural Networks (ML/CNN) can help you in any way!

Well, at the very least we will try to help you sort your data and find on it the earthquakes that you will need in your research as a seismologist.

DASight, created as part of the TRANSFORM^2 project, is little software that allows to detect earthquakes in DAS data. It was originally conceived to aid in the task of selecting events during continuous DAS recordings. The main idea is to generate little figures from the data and select which files contain events and which are just noise. Then the files can be saved in different directories, or deleted (in the case of noise files). But, there is a few more things that DASight can do!

We hope that this can catch your attention. Happy reading and classification!

1. WELCOME TO DASIGHT

Everyone knows that Distributed Acoustic Sensing (DAS) systems generate extremely large continuous datasets, often exceeding one terabyte per day. This poses a big challenge for storing, transferring and processing the recorded information. In many monitoring scenarios, only a small fraction of the recorded data contains seismic signals of scientific interest, while the majority corresponds to background noise. DASight has been developed to address this challenge! It enables "smart" and automated data selection based on image-based machine learning.

The main idea behind DASight is that one can determine whether a given DAS data segment (file) contains a seismic event or not, something that, to the trained human eye is quite clear. Therefore, the software focuses on a binary classification problem (event vs. noise) rather than more detailed processing steps such as picking, localization, or magnitude estimation. This choice allows DASight to be a featherweight tool to reduce data volumes while preserving potentially relevant recordings.

DASight is mainly aimed at near-real-time monitoring, where DAS files are screened as they are recorded by an acquisition system. It is built to aid in situations where only event-related data are stored for further analysis. Nevertheless, it can be used on archived DAS datasets to identify event-containing segments.

Finally, we have tried our best to design DASight to be easily integrated into existing DAS processing frameworks. It operates through file-based communication and can be called by simple shell (BASH) scripts. The software builds upon DASvader.jl (https://github.com/marianoarnaiz/DASvader.jl), an open-source Julia package developed at IPGP for DAS data handling and signal processing, and extends it with image generation and machine-learning-based classification capabilities. DASight can function as a standalone screening tool or as a front-end module feeding downstream seismic analysis pipelines

2. WHAT IS DASIGHT AND WHO IS IT FOR?

- What is DASight?

As explained above, DASight is a software developed for the classification of DAS data. In essence, DASight is a series of codes and scripts. The core processing tools were developed in Julia programming language (no, its not Python), leveraging its speed, strengths in numerical computing, signal processing, and machine learning. The managing of files is performed by a series of BASH scripts (this provides flexibility and ease of use). No dedicated virtual environment is required beyond the standard Julia package manager and BASH and a full install_DASight.sh script is provided so everything (hopefully) runs smoothly. Once the package is properly installed, you should be able to test several examples/way-of-use that we have included.

The software is designed and implemented in macOS, but it runs on Linux-based systems; compatibility with Windows is not tested.

- Who is it for?

Of course! DASight was designed for seismologists. If you have an optical fiber attached to a DAS interrogator and you are recording earthquakes, this software is for you! But, if you have older data and you would like some ML eye on it, maybe it can help you too! And there is more: the same ideas can be applied to other types of monitoring (glaciers, cities, buildings, etc.), or to classify your data in more than two categories (not only event/noise). For that you need some preliminary (training) data and a re-training of the networks. A guide to RE-TRAIN the CNN is on our to-do list (see Chapter 10), so stay tuned.

- What DASight is not!

I know it is unconventional to add a "what is it not" section to the manual of a software, but I find it today's world we all want fast solutions to everything. So, here it is. DASight is not an advanced processing toolkit for DAS data, if you need to process DAS data (say, as SAC does for single station data) go to the DASvader.jl github (https://github.com/marianoarnaiz/DASvader.jl), hopefully it will help you. Moreover, we have not created this code to perform a lot of other ML based steps (denoise, picking and what not), it is just an observation tool: Can I see an event in the data? Yes/No. Finally, we have taken it to heart to keep a light-weight code, meaning that a lot is purposely not-done (we just filter and plot the data). If you like advance processing steps, you

are very welcome to add them to the workflow. Finally, DASight is not a ML tutorial in any way shape or form, but it might help someone starting to dip their toes in this waters, just tread carefully.

3. DASIGHT INSTALLATION

OK, after all of that, let's begin!

- Where to download?

Currently we share DASight via GitHub:

https://github.com/marianoarnaiz/DASight.jl

The big files (the trained models and the example data) do not fit in GitHub, so they live in Zenodo:

https://zenodo.org/records/24109156

All you need should be there. You do not have to download the Zenodo files by hand: Julia fetches them for you the first time they are needed (see Chapter 7).

- Install script

We provide a basic script to install DASight, called `install_DASight.sh`. It installs juliaup (the tool that installs and updates Julia), the current version of Julia, the packages DASight needs (including the three tricky ones that are not in Julia's public registry) and DASight itself. At the end it checks that DASight loads. You need macOS or Linux, an internet connection, and the commands `curl` and `git`. Download the script from GitHub, open a terminal in the folder where you saved it and run:

```
bash install_DASight.sh
```

Or skip the download and run it straight from GitHub:

```
curl -fsSL https://raw.githubusercontent.com/marianoarnaiz/
DASight.jl/main/install_DASight.sh | bash
```

Go and make some coffee: the first time, Julia compiles a big plotting and machine-learning stack, and that takes several minutes. When it finishes, jump to "Get the examples" below. If you prefer to do everything yourself, keep reading.

- Installation by hand?

To those who love a little challenge and are curious about Julia, I leave here the step by step instructions to be able to install all DASight dependencies. After all is installed, just downloading the scripts should be enough to use DASight.

- Install Julia Language

Installing Julia is rather simple. First be sure to check the download section of Julia's website:

https://julialang.org/downloads/

If nothing has changed, running this command should be enough (it installs Julia in your home folder):

```
curl -fsSL https://install.julialang.org | sh
```

If that works, you should be able to launch Julia from any terminal by typing "julia" and hitting the return button.



Fig 1. Initialize Julia from Linux terminal

- Installing required dependencies and packages

Installing packages inside Julia is rather simple, but it takes some time. Start Julia and hit the "]" key in your keyboard to get into the "package managing mode" or pkg. There, create a shared environment called DASight. Think of it as a little room just for DASight, so it does not mix with your other Julia projects:

```
activate --shared DASight
```

Now we add DASight and the three packages it needs that are not in Julia's public registry: DASvader.jl and its two helpers, Geodesics.jl and Seis.jl. Everything goes in one single command (all in one line!):

```
add https://github.com/anowacki/Geodesics.jl https://github.com/anowacki/
Seis.jl https://github.com/marianoarnaiz/DASvader.jl https://github.com/
marianoarnaiz/DASight.jl
```

That's it! All the other packages in Table 1 are installed for you, so no more adding them one by one (that was the old way, sorry). The first install, and the first `using DASight`, take a while because Julia is compiling. If the line above does not work, copy the one in the DASvader.jl GitHub page (https://github.com/marianoarnaiz/DASvader.jl).

Table 1: What DASight uses under the hood (installed for you).

Category	Packages
DAS processing	DASVader.jl, Geodesics.jl, Seis.jl, NCDatasets.jl, StatsBase.jl
Time handling	Dates.jl, NanoDates.jl
Imaging and Image Processing	CairoMakie.jl, Images.jl, ImageIO.jl, FileIO.jl
Machine learning	Flux.jl, Metalhead.jl, MLUtils.jl
Model storage	BSON.jl
Standard libraries	Random.jl, Statistics.jl
Signal processing	FFTW.jl, DSP.jl, ImageFiltering.jl
File handling	JLD2.jl, DelimitedFiles.jl

WARNING: A word of caution!

At the moment of development of DASight and the writing of this manual, Metalhead.jl (the package that provides machine learning vision models for use with Flux.jl, Julia's main ML ecosystem) is not updated to work with the most current version of Flux.jl. Good news: DASight already asks for the right versions (Flux

0.14.25, Metalhead 0.9.5 and MLUtils 0.4.5), so you do not need to do anything. Just be careful if you add other packages to the same environment and Julia offers to "upgrade" those three: say no! If you ever need to put them back by hand, these are the lines:

```
using Pkg
Pkg.add(Pkg.PackageSpec(name="Metalhead", version="0.9.5"))
Pkg.add(Pkg.PackageSpec(name="MLUtils", version="0.4.5"))
Pkg.add(Pkg.PackageSpec(name="Flux", version="0.14.25"))
```

- Get the examples

Back in the normal Julia prompt (press backspace to leave package mode), type:

```
using DASight
DASight.install_examples("~/DASight_examples")
```

This copies the three examples (`1_PAINTER`, `2_DETECTIVE` and `3_WATCHER`, each with its launcher scripts) and a small example data set into a folder that you can edit and play with. The trained models (a few hundred MB) are downloaded automatically, once, the first time they are used.

- Does it work? A quick test

Open a terminal and run the first two steps of the pipeline on the example data:

```
cd ~/DASight_examples/1_PAINTER
./run_painter.sh data_test
cd ../2_DETECTIVE
./run_detective.sh ../1_PAINTER/DASight_pics
```

If pictures appear in `DASight_pics` and the lists of signals and noise appear in `DASight_results`, you are all set. If a launcher says permission denied, run `chmod +x *.sh` in that folder once (or start it with `bash run_painter.sh`). The launchers look for DASight in the `@DASight` environment and then in Julia's default one; to use another one, start them like this: `DASight_ENV=/path/to/env ./run_painter.sh ...`



4. PAINTER: GETTING DATA IN AND MAKING IMAGES

The first step in using DASight is to be able to load data for it. This might be a bit confusing at first as there is not a universal DAS data format and the users must adapt the data-reading routine to their own acquisition system and file format. The rest of the image-processing pipeline can remain unchanged, provided that the data are correctly loaded. For DASight, raw DAS records must be converted into standardized 224 × 224 pixel PNG images. These images required by the machine-learning classifier and are also used to build the training dataset.

The directory **1_PAINTER** includes a Bash and Julia-based pipeline that: reads DAS records, performs the necessary signal processing, and generates the corresponding images.

4.1. The DAS data structure

DASight uses the iDAS data structure provided by DASVader.jl. The format is rather simple and only requires key information, namely the strain-rate matrix and the time and distance vectors. Regardless of the original data format, the information must be organized into the following object:

```
dDAS = iDAS(data, timee, htime, offset, empty_attb(), filename)
```

The required arguments are:

Argument	Description
`data`	DAS data matrix, organized as **time samples × channels**. Each column represents one DAS channel, and each row represents one time sample.
`timee`	Time axis in seconds, starting at 0.0, with a constant sampling interval (`dt`). Its length must match the number of rows in `data`.
`htime`	Human-readable time axis (`DateTime`), corresponding to the absolute time of each sample. Its length must also match the number of rows in `data`.
`offset`	Spatial axis in metres, describing the position of each DAS channel along the fibre. Its length must match the number of columns in `data`.
`empty_attb()`	An empty metadata structure required by `iDAS`. DASight does not use these attributes, so they can remain empty.
`filename`	String containing the name of the original data file.

The most important requirement is the orientation of the data matrix:

```
size(data) = (number_of_time_samples, number_of_channels)
```

For example, a record containing 10,000 time samples and 500 DAS channels must be organized as a 10,000 × 500 matrix.

The time and spatial axes must be consistent with the matrix dimensions:

```
length(timee) == size(data, 1)
length(htime) == size(data, 1)
length(offset) == size(data, 2)
```

The time axis should start at zero and increase according to the sampling interval. The spatial axis should describe the position of each channel along the fibre, using the appropriate channel spacing.

The `empty_attb()` function simply creates an empty metadata object. It is included because the iDAS structure requires it, not because DASight needs additional metadata to perform the image processing. Notice that the attributes array may hold meta data information for you, but it is not required to process the data.

In summary: The only requirement for integrating a new DAS data format into DASight is to provide a function that reads the original file and returns a correctly assembled iDAS object. Once this is achieved, the remaining processing functions can be used without modification.

4.2. Reading DAS records: the NetCDF example

The supplied helper script, DAS2Pics_nc.jl, provides an example of how to read DAS records stored in NetCDF (.nc) format. The function read_das_nc() extracts the DAS matrix, time axis, and spatial axis from the input file and organizes them into the required iDAS structure.

At the beginning of the script, users will find a SETTINGS section:

```
# Names of the variables inside your .nc files
const VAR_DATA   = "StrainRate"
const VAR_OFFSET = "offset"
const VAR_TIME   = "time"
# Time unit of VAR_TIME
const TIME_UNIT  = nothing
```

These settings must be adapted to match the variable names and time units used in the user's NetCDF files.

When TIME_UNIT = nothing, the script attempts to identify the time unit from the NetCDF variable's units attribute. Supported units are:
Nanoseconds
Microseconds
Milliseconds
Seconds

If the time unit cannot be identified automatically, it must be specified manually.

The `read_das_nc()` function also checks the orientation of the original data matrix and transposes it when necessary to obtain the required time × channels convention.
Adapting the reader to other data formats

The NetCDF reader is provided as an example, not as a universal DAS reader. Users working with other formats (HDF5, SEG-Y, proprietary interrogator formats, etc.)

should modify or replace the `read_das_nc()` function accordingly. **The authors are happy to help with any problems that arise while importing data to DASight.**

Only this initial data-reading step needs to be adapted. The resulting data must simply be assembled into a valid iDAS object.

For users working with FEBUS A1 interrogators, data can be loaded directly using the `rdas()` functionality available in DASVader.jl, which simplifies the process of reading and organizing the records.

Before proceeding, the script calls:

```
check_das(dDAS)
```

This function performs basic consistency checks on the data, including matrix dimensions, time and spatial axes, sampling interval, channel spacing, NaN/Inf values, and empty channels. It also reports a summary of the record.

4.3. Generating the PNG images

Once the data-reading function is properly configured, users do not need to interact with the image-processing functions individually.
DASight provides a Bash launcher, run_painter.sh, which automatically calls the Julia helper script and executes the complete image-generation pipeline.
The pipeline performs the following operations:

- Data preparation: Removes the mean and linear trend, normalizes the data, and handles invalid values.
- Automatic frequency-band selection: Estimates the dominant frequency band using the power spectral density (PSD).
- Band-pass filtering: Applies a fourth-order band-pass filter using the selected frequency limits.
- Frequency-wavenumber analysis: Identifies the dominant frequency-wavenumber peak in the record.
- Gaussian filtering: Applies a Gaussian filter whose width is determined by the dominant FK peak.
- Image generation: Converts the processed DAS matrix into a 224 × 224 pixel heatmap.
- Contrast adjustment: Applies automatic intensity adjustment to improve image
- contrast.

- The final images are saved as PNG files with the suffix _AUTO.png.

4.4. Running the PAINTER

Before running the script, make sure that:
- Julia is installed and accessible from the terminal.
- The required Julia packages are installed.
- The DAS2Pics_nc.jl script and its associated Image-Signal-Processing.jl file are in their expected locations.
- The input directory contains the DAS records to be processed.
- The data-reading routine has been configured for the input file format.

The first time the Bash script is used, it may be necessary to grant execution permissions:

chmod +x run_painter.sh

The script can then be executed directly from the terminal.

Default execution

./run_painter.sh

By default, the script reads all .nc files in the data_test directory and saves the resulting PNG images in a directory called DASight_pics.

Specify an input directory

./run_painter.sh /path/to/my_nc_files

Specify both input and output directories

./run_painter.sh /path/to/my_nc_files /path/to/output_images

Keep the original images

./run_painter.sh data_test DASight_pics --keep-originals

By default, the intermediate PNG images are removed after contrast adjustment, leaving only the final _AUTO.png images. Use --keep-originals to retain both versions.

4.5. Output

After successful execution, the output directory will contain one final PNG image for each successfully processed DAS record. These images are standardized at 224 × 224 pixels and are ready to be used by the DASight classification and training workflows. The PAINTER therefore acts as the interface between the original DAS acquisition data and the image-based machine-learning components of DASight.

Remember: The user only needs to adapt the data-reading function to their own DAS format and run the Bash launcher. The subsequent processing and image-generation steps are handled automatically by DASight.



5. DETECTIVE: CLASSIFICATION OF AN ARCHIVE

Classifying DAS images and identifying potential events

Once the DAS records have been converted into PNG images using the PAINTER module, the next step is to classify them and identify potential seismic events.

DASight uses three independently trained ResNet models to distinguish between signal and noise:

- **ZEUS**
- **POSEIDON**
- **HADES**

The DETECTIVE module applies all three models to every image in the input directory. Their predictions are then combined using a three-stage decision scheme to produce a final classification.

The objective is to reduce false detections by combining the predictions of multiple models rather than relying on a single classifier.

5.1. Input data and trained models

The input to DETECTIVE is a directory containing the PNG images generated by PAINTER. Each image is processed independently by the three trained neural networks.

The three models are BSON files: `ZEUS.bson`, `POSEIDON.bson` and `HADES.bson`. You do not need to download them by hand: the first time they are needed, DASight fetches them from Zenodo (a few hundred MB, only once) and keeps them in Julia's cache. If you prefer your own copies (for example to work offline), set the environment variable `DASight_MODELS_DIR`, or give a models folder as the third argument of `run_detective.sh`.

The supplied Julia script, `Pics2Events.jl`, automatically loads these models and performs the classification.

By default, DETECTIVE looks for images in the `DASight_pics` directory and uses the models that DASight downloads for you.

Users can change these locations through the command-line arguments or modify the SETTINGS section of the Julia script.

5.2. The three-stage decision scheme

Each image is evaluated by all three neural networks. Their predictions are combined through a three-stage voting scheme.

The default configuration is defined in the SETTINGS section of `Pics2Events.jl`:

```
const PARAMS = PipelineParams(
    hades_noise_threshold = 0.95,
    score_threshold       = 0.5,
    min_signal_votes      = 2,
    w_zeus                = 1.1,
    w_poseidon            = 0.4,
    w_hades               = 1.5
)
```

The decision process consists of three stages:

Stage 1 – HADES noise rejection

HADES acts as an initial noise filter. If it identifies an image as noise with a probability greater than the specified threshold (0.95 by default), the image is discarded as noise.

Stage 2 – Weighted score

For images that pass the first stage, the predictions of the three models are combined into a weighted score.

Each model contributes according to its assigned weight:

Model	Weight
ZEUS	1.1
POSEIDON	0.4

HADES	1.5

The resulting score must exceed the specified threshold (0.5 by default) for the image to qualify as a signal candidate.

Stage 3 – Minimum signal votes

Finally, the number of models identifying the image as signal is evaluated.

At least two models must vote for signal (`min_signal_votes = 2`) for the image to be classified as a signal.

The three stages work together to provide a more conservative classification than relying on a single model.

All thresholds and weights can be adjusted in the `PARAMS` section of `Pics2Events.jl`.

5.3. Running the DETECTIVE

The classification process is launched using the Bash helper script, `run_detective.sh`.

Before running it for the first time, make sure that:

- Julia is installed and accessible from the terminal.
- The required Julia packages are installed.
- The three trained models are available (DASight downloads them for you the first time, so you need an internet connection once).
- The input directory contains the PNG images to be classified.
- The `Pics2Events.jl` script and its associated `ML_Handling.jl` file are in their expected locations.

Grant execution permissions if necessary:

```
chmod +x run_detective.sh
```

The script can then be executed directly from the terminal.

Default execution

```
./run_detective.sh
```

By default, the script reads images from `DASight_pics`, loads the three trained models (downloaded for you the first time) and writes the results into `DASight_results`.

Specify an input directory

```
./run_detective.sh Long_Test
```

This classifies all images found in the `Long_Test` directory and saves the results in the default output directory.

Specify both input and output directories

```
./run_detective.sh Long_Test my_results
```

Specify a custom models directory

```
./run_detective.sh Long_Test my_results /path/to/MODELS
```

The script automatically checks that Julia is available and that both the input and model directories exist before starting the classification.

5.4. Output files

After classification, DETECTIVE generates five text files in the output directory.

The filenames include a tag corresponding to the name of the input image directory.

Output file	Description
`Results_<tag>.txt`	Individual classification and probability reported by each of the three models.
`pipeline_output_<tag>.txt`	Combined classification results, including scores, votes, and the final decision for each image.
`found_signals_<tag>.txt`	List of images classified as SIGNAL.
`found_noise_<tag>.txt`	List of images classified as NOISE.
`others_<tag>.txt`	List of images for which the models disagreed.

These files provide both the final classification and the information needed to inspect the individual model predictions.

The `found_signals` and `found_noise` files can be used directly to identify the images selected for further analysis, while `others` provides a convenient way to identify ambiguous cases that may require additional inspection.

5.5. Additional settings

Several parameters can be modified in the SETTINGS section of `Pics2Events.jl`.

Image selection

```
const NAME_CONTAINS = ""
```
By default, all images found in the input directory are classified.

If the directory contains both original and contrast-stretched images, users can restrict the classification to files containing a particular string. For example:

```
const NAME_CONTAINS = "_AUTO"
```
This is useful when the directory contains both versions of the PAINTER output.

Batch processing

```
const BATCHSIZE = 16
```
Controls the number of images processed in each batch. Reduce this value if the available memory is insufficient.

Terminal output

```
const SHOW_TABLE = false
const ALT_SCORES = false
```
`SHOW_TABLE` enables a per-image summary in the terminal, while `ALT_SCORES` enables an additional legacy score summary.

5.6. Final remarks

DETECTIVE is the classification component of DASight. It receives the standardized images generated by PAINTER, evaluates them using three trained neural networks, and combines their predictions to identify potential seismic events.

The classification results are saved as text files, allowing users to inspect the individual model predictions, identify detected signals, and review cases where the models disagree.

In summary: Once the input images and trained models are available, the user only needs to run `run_detective.sh`. The classification, voting procedure, and generation of the output files are handled automatically by DASight.



6. WATCHER: LIVE RECORDING CLASSIFICATION

PAINTER and DETECTIVE are great when you already have a pile of files. But what if the files keep arriving, say one every minute, from an interrogator that never sleeps? That is the job of the WATCHER. Think of it as a very patient colleague sitting next to your recording folder: every time a new DAS file shows up, it paints the picture, asks ZEUS, POSEIDON and HADES what they think, and files the data away. Signals are always kept; what happens to the noise is up to you.

The models are loaded only once and stay in memory, so you wait for them just at the start. The decision scheme (weights, thresholds and votes) is meant to be exactly the same as in DETECTIVE (see Section 5.2), so both should give the same answers.

6.1. Preparing your directory and data

The WATCHER looks at one folder: `DAS_RECORDING` by default, or any folder you give it. Your acquisition system (or a script of yours) just has to drop the DAS files there.

The example reads `.jld2` files (the DASvader format) that contain an iDAS object stored under the name `dDAS`. These and a few other things live in the SETTINGS section of `Watch_DAS.jl`:

```
const FILE_EXT      = ".jld2"
const DAS_VARNAME   = "dDAS"
const POLL_SECONDS   = 5
const MIN_FILE_AGE   = 2
```

- `FILE_EXT` and `DAS_VARNAME`: which files to process, and the name of the variable inside them.
- `POLL_SECONDS`: how often (in seconds) the WATCHER looks for new files.
- `MIN_FILE_AGE`: files modified in the last N seconds are ignored, because they are probably still being written.

Files whose name starts with a dot are ignored too. So, if you write your own recorder, copy each file under a hidden name (for example `.myfile.part`) and rename it when it is complete. This is what `start_recording.sh` does, so the WATCHER never sees half-written files.

Different data format? Just like in the PAINTER, only the reading step changes: it has to return a valid iDAS object (see Section 4.1). The authors are happy to help.

Inside the watched folder, the WATCHER creates these folders and files:

Name	What it is
`SIGNALS/`	Files with signals. Always kept.
`NOISE/`	Noise files kept by the policy you choose with `--keep-noise` (see below).
`FAILED/`	Files that could not be read or processed.
`PICS/`	The pictures. Only kept if you use `--keep-pics`, otherwise they are deleted after use.
`watcher.log`	One line per file: time, file, decision, score, votes and what was done with it.

Every new file is moved out of the watched folder, so `DAS_RECORDING` itself stays almost empty.

6.2. Important parameters

What to do with the noise is decided when you start the WATCHER, with the option `--keep-noise`:

Option	What happens to noise files
`--keep-noise 1`	Keep all of them. This is the default.
`--keep-noise 10`	Keep 1 of every 10.
`--keep-noise 100`	Keep 1 of every 100.
`--keep-noise 0`	Keep none: noise files are **deleted**.

WARNING: with `--keep-noise` other than 1, data is permanently deleted. Please try it first on a copy of your data!

There are a few more options:

Option	What it does
`--keep-pics`	Keep the pictures in `PICS/`.
`--models DIR`	Use your own folder with the `.bson` models instead of the ones DASight downloads.
`--once`	Process the files that are waiting and exit. Handy for testing.

The 1-in-N count continues across restarts, so stopping and starting the WATCHER does not reset your noise policy. The weights and thresholds of the decision scheme are in the SETTINGS of `Watch_DAS.jl` (the `PARAMS` block); keep them equal to the ones in `Pics2Events.jl` unless you have a good reason.

6.3. Running the WATCHER: a test drive

No instrument at hand? No problem, we include a fake one. You need two terminals, both in the `3_WATCHER` folder. The first time only, grant execution permissions:

```
chmod +x run_watcher.sh start_recording.sh
```

In the first terminal, start the fake instrument. It copies one of the example files into `DAS_RECORDING` every 10 seconds:

```
./start_recording.sh Fake_DAS DAS_RECORDING 10
```

In the second terminal, start the WATCHER:

```
./run_watcher.sh DAS_RECORDING --keep-noise 10
```

After loading the models, you will see the new files being classified one by one and moved away: signals to `SIGNALS/`, noise to `NOISE/` (1 out of every 10 in this example, the rest are deleted), and unreadable files to `FAILED/`. Every decision is added to `watcher.log`. Stop both terminals with Ctrl+C.

The arguments of `start_recording.sh` are the source folder (default `Fake_DAS`), the output folder (default `DAS_RECORDING`) and the seconds between files (default 60). And `./run_watcher.sh` alone watches `./DAS_RECORDING` and keeps all the noise.

6.4. Using it for real

When you are happy with the test drive, point the WATCHER at the folder where your acquisition system writes its files:

```
./run_watcher.sh /path/to/your/recordings --keep-noise 10
```

It is meant to run for days, so start it inside `tmux` or `screen`: it will survive when you close the terminal. If you only want to clean up a folder that was left unattended, use `--once`.

In summary: start your instrument (or the fake one), run `run_watcher.sh` with the noise policy you like, and check `SIGNALS/` and `watcher.log` in the morning.

7. MODELS AND EXAMPLE DATA

The trained models and the example data are too big for GitHub, so they live in Zenodo (https://zenodo.org/records/24109156). You do not have to download them yourself: Julia does it for you the first time they are needed, checks the files against a checksum (so you know nothing got corrupted on the way) and keeps a copy in its cache, in `~/.julia/artifacts`. So it only happens once.

Zenodo file	What is inside
`DASight_models.tar.gz`	The three trained networks: `ZEUS.bson`, `POSEIDON.bson` and `HADES.bson` (about 376 MB).
`DASight_example_data.tar.gz`	The example data: `data_test/` (`.nc` files, used by the PAINTER) and `Fake_DAS/` (12 `.jld2` files, used by the WATCHER). About 291 MB.

7.1. Asking for them from Julia

In Julia, these two functions return the path of a folder with the files. The first call downloads them:

```
using DASight
models_dir()
example_data_dir()
```

You will see a message like "Getting the DASight models..." during the first download. You may also see a line saying "Failure artifact" just before "Downloaded artifact". Do not worry: Julia first looks for the files in its own package server, does not find them there, and then falls back to Zenodo. That is exactly what we want.

7.2. Using your own copies (or working offline)

If you prefer to use your own copies, for example because you are offline, tell DASight where they are with these two environment variables:

```
export DASight_MODELS_DIR=/path/to/MODELS
export DASight_DATA_DIR=/path/to/example_data
```

When they are set, DASight does not download anything.

8. USING DASIGHT AS A LIBRARY

If you prefer Julia scripts to Bash launchers, everything the launchers do is also available as functions. A small example, from data to decision:

```
using DASight

# data -> picture
dDAS = read_das_nc("myfile.nc")
das2pics(dDAS; label="myfile.nc", outdir="pics")
batch_auto_level_intensity(dir="pics",
remove_original=true)

# pictures -> decision
models = load_detective_models(models_dir(),
                (zeus="ZEUS.bson", poseidon="POSEIDON.bson",
hades="HADES.bson"))
results,   decision   =   detect_files(list_images("pics"),
models)
#  decision[:,  1]  =  file,  decision[:,  5]  =  "SIGNAL"  /
"NOISE"
```

Every function has a docstring: type `?` in Julia followed by the name, for example `?das2pics`, `?run_pipeline` or `?watch`. Only the functions that DASight exports are found like that. For the internal ones, put `DASight.` in front: `?DASight.median_psd_welch`.

9. WHEN THINGS GO WRONG

Things that happened to us (and to our friends), and what to do about them:

What you see	What to do
permission denied	Run `chmod +x *.sh` in that folder once, or start the launcher with `bash`, for example `bash run_painter.sh`.
the Julia package DASight was not found	Install DASight in the shared environment (Chapter 3), or tell the launcher where it lives: `DASight_ENV=/path/to/env ./run_painter.sh ...`
expected package DASVader to be registered	DASvader, Geodesics and Seis are not in Julia's public registry. Add all of them from their GitHub addresses, in the same command (Chapter 3).
The first run takes forever	Normal! Julia is compiling a big plotting and machine-learning stack. Be patient.
The models or the example data do not download	Check your internet connection and try again, or use your own copies (Section 7.2).
Windows	We have not tested it. Sorry!

Still stuck? The authors are happy to help. Open an issue at https://github.com/marianoarnaiz/DASight.jl/issues and tell us what you see.

10. WHAT'S NEXT, AND HOW TO CITE

DASight is young (version 0.1) and we have plans:

- an HDF5 (`.h5`) reader, next to the NetCDF one;
- tiling of very large files into windows before painting them;
- a guide (and scripts) to RE-TRAIN the networks, for other kinds of events or other kinds of monitoring;
- Windows testing, and registration in Julia's General registry once DASvader is registered.

Issues and suggestions are very welcome.

Citing DASight

If you use DASight in your work, please cite it. The file `CITATION.cff` in the repository has all the details, and the "Cite this repository" button on GitHub uses it.

Authors and license

DASight was written by Mariano S. Arnaiz-Rodríguez and Claudio Satriano (Université Paris Cité, Institut de physique du globe de Paris, CNRS), as part of the TRANSFORM² project. It is free software, released under the MIT license.

Thanks for reading, and happy classification!